



UNIVERSIDAD CENTROCCIDENTAL "LISANDRO ALVARADO"
DECANATO DE CIENCIAS Y TECNOLOGÍAS
BARQUISIMETO, ESTADO LARA



GAME CONNECT

EQUIPO 1 - PRIMER ENTREGABLE

INTEGRANTES

29.654.138 Adrián Pereira

25.149.300 Dehucarlys Azuaje

19.241.241 Edgar López

28.316.086 Hanuman Sanchez

27.524.130 José Alvarado

Asignatura: Laboratorio III

Prof: Jorge Chiquín

Barquisimeto, Abril de 2026

Descripción general de la aplicación.....	3
Alcance.....	3
Objetivo.....	3
Entregables (Funcionalidades por Módulo).....	3
Limitaciones.....	4
Criterio de Éxito.....	4
Listado de requisitos.....	5
Requisitos Funcionales.....	5
Requisitos No Funcionales.....	6
Modelo entidad relación (MER).....	7
Herramientas tecnológicas.....	8
Transversales: Git y Github.....	8
Cliente Móvil: React Native con Expo.....	8
API: Nestjs.....	8
Websockets: Socket.io.....	9
Bases de Datos: PostgreSQL, Meilisearch.....	9
Cache: Valkey.....	9
Almacenamiento de multimedia: Cloudinary.....	9
Otros servicios: IGDB, Expo FCM notifications, Resend.....	10
Arquitectura de la aplicación.....	10
Prototipo visual.....	12

Descripción general de la aplicación

La plataforma consiste en una red social especializada en el ecosistema de los videojuegos, diseñada para conectar a jugadores, permitir el descubrimiento de nuevos títulos y facilitar la interacción comunitaria en tiempo real. A diferencia de las redes sociales genéricas, esta aplicación centra la experiencia en el perfil del jugador y su relación con una base de datos global de títulos, permitiendo a los usuarios organizar su historial de juegos, compartir reseñas y participar en chats dinámicos.

Alcance

Objetivo

Desarrollar una plataforma de red social modular orientada a entusiastas de los videojuegos, que permita la interacción en tiempo real, la gestión de perfiles de juegos y la creación de comunidades mediante una arquitectura escalable y de alto rendimiento.

Entregables (Funcionalidades por Módulo)

- Módulo de Usuario y Auth: Sistema de registro, inicio de sesión (JWT) y perfiles personalizables con avatares gestionados en Cloudinary.
- Módulo de Games (Integración IGDB): Buscador de videojuegos con Meilisearch, visualización de fichas técnicas, sistema de "favoritos" y reseñas/reviews.

- Módulo de Posts y Feed: Creación de publicaciones con soporte multimedia (imágenes), sistema de "likes" y comentarios, con un feed relacional servido desde PostgreSQL.
- Módulo de Chat y Notificaciones: Chat privado en tiempo real mediante WebSockets (Socket.io) con gestión de estados de conexión y notificaciones push (Expo FCM) para interacciones clave.
- Cliente Móvil (Android): Aplicación funcional desarrollada en React Native/Expo con navegación fluida y consumo de la API modular.
- Infraestructura Backend: Servidor NestJS con arquitectura de Monolito Modular, base de datos relacional (PostgreSQL), motor de búsqueda (Meilisearch) y capa de caché/colas (Valkey + BullMQ).

Limitaciones

- Contenido Externo: El proyecto no incluye la creación de la base de datos de juegos; se depende exclusivamente de la API de IGDB.
- Despliegue Web: El MVP se limita estrictamente a la aplicación móvil (APK/IPA); no se incluye versión de escritorio o web en esta fase.
- Moderación Automática: No se implementarán sistemas de IA para detección de contenido ofensivo en imágenes o texto (moderación manual).
- Pasarela de Pagos: No se incluyen funcionalidades de monetización, suscripciones o compras in-app en la versión inicial.

Criterio de Éxito

- Rendimiento de Búsqueda: Las consultas en el módulo de *Games* vía Meilisearch deben arrojar resultados en menos de 100ms.
- Latencia de Chat: Los mensajes enviados por WebSockets deben ser entregados al destinatario en menos de 1 segundo en condiciones normales de red.
- Disponibilidad de Datos: El sistema de caché (Valkey) debe reducir las consultas directas a PostgreSQL para perfiles y feed en al menos un 40%.
- Compilación e Integración: Superar con éxito las pruebas de compilación en la nube mediante EAS y asegurar que el 100% de los endpoints REST estén documentados y funcionales.

Listado de requisitos

Requisitos Funcionales

- RF-01: El sistema deberá permitir el registro, inicio de sesión y cierre de sesión de usuarios mediante correo electrónico y contraseña.
- RF-02: El sistema deberá permitir que los usuarios creen una cuenta o inicie sesión utilizando sus credenciales de Twitch, Google, o Steam.
- RF-03: El sistema deberá permitir al usuario la edición de su perfil.
- RF-04: El sistema deberá mostrar los perfiles de otros usuarios, incluyendo su historial de publicaciones.
- RF-05: El sistema deberá permitir al usuario crear publicaciones de texto.
- RF-06: El sistema debe permitir al usuario seguir y dejar de seguir a otros usuarios.

- RF-07: El sistema deberá permitir al autor de una publicación editar o eliminar su contenido en un plazo de 24 horas.
- RF-08: El sistema deberá permitir a los usuarios interactuar con las publicaciones mediante "likes" y comentarios.
- RF-09: El sistema deberá mostrar un feed o línea de tiempo global con las publicaciones más recientes de todos los usuarios.
- RF-10: El sistema deberá ofrecer un buscador que filtre publicaciones por etiquetas, nombres de juegos o palabras clave en la descripción.
- RF-11: El sistema deberá permitir al usuario guardar publicaciones en una sección de "Favoritos" para su consulta posterior.
- RF-12: El sistema deberá facilitar el envío y recepción de mensajes de texto privados entre múltiples usuarios en tiempo real.
- RF-13: El sistema debe ofrecer un indicador visual de espera con acciones que tomen más de 1 segundo.

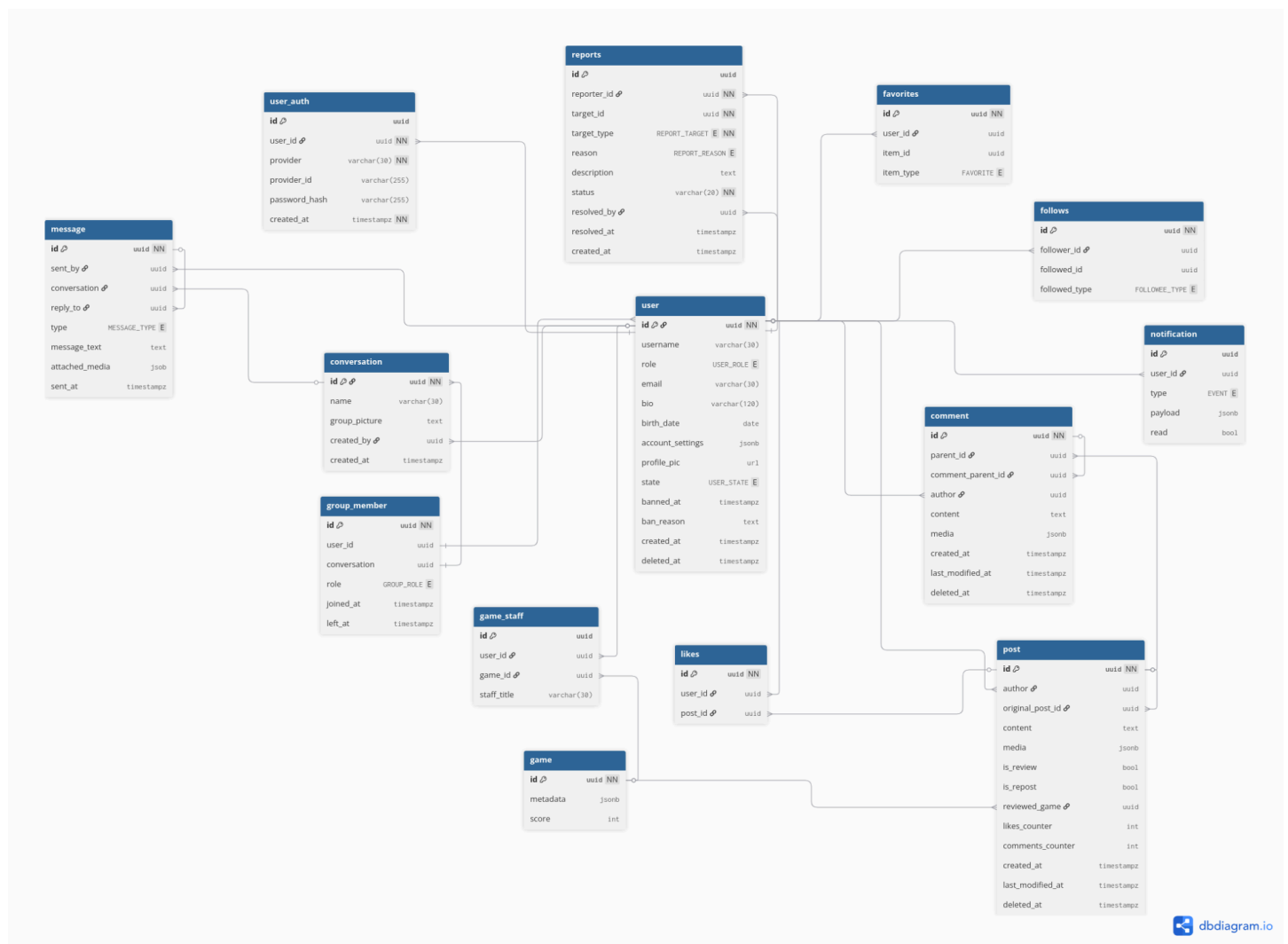
Requisitos No Funcionales

- RNF-01: El sistema deberá entregar el feed de publicaciones, y resultados de búsqueda con respuestas de baja latencia.
- RNF-02: La interfaz de usuario deberá adaptarse automáticamente a los diferentes tamaños de pantalla.
- RNF-03: La interfaz de usuario deberá mantener un patrón gráfico común en todas las pantallas, siguiendo la guía de diseño establecida.
- RNF-04: La interfaz de usuario debe ser intuitiva y amigable.

- RNF-05: El sistema debe ser compatible con las versiones de Android 5 en adelante.
- RNF-06: El sistema deberá validar todas las entradas de datos.
- RNF-07: El sistema debe estar estructurado de forma modular para permitir la actualización de sus componentes, sin afectar el funcionamiento del resto de la plataforma.
- RNF-08: El sistema deberá encriptar las credenciales de acceso.

[nota: revisar anotaciones sobre requisitos aca](#)

Modelo entidad relación (MER).



[\(Ver diagrama MER en dbdiagram.io\)](#)

Herramientas tecnológicas

Para esta plataforma se ha optado por una arquitectura Cliente - Servidor más un Conjunto de Bases de Datos Relacionales y No Relacionales. Usando los protocolos REST y Websockets para la interacción entre ambos extremos. El análisis de los requisitos nos permite decidir qué tecnología utilizar para cada característica o grupo de características. El backend adopta el patrón de Monolito Modular: un único proceso desplegable internamente organizado en módulos (Auth, Posts, Users, Games, Feed, Chat, Search, Notifications). Esto elimina la complejidad operativa de los microservicios, manteniendo a la vez una separación de responsabilidades que permite extraer módulos a servicios independientes si el crecimiento lo justifica en el futuro.

Transversales: Git y Github

Git y GitHub serán la plataforma central para la gestión colaborativa del código, permitiendo sincronizar contribuciones mediante un flujo estructurado de ramas que asegura trazabilidad e integridad.

Cliente Móvil: [React Native](#) con [Expo](#).

Escogemos RN por dos razones precisas, la primera es el dominio de JS/TS y Reactjs en todo el equipo de desarrollo, y además, es accesible el enorme ecosistema ya disponible para frontend web con Reactjs (Zustand, Tan Stack Query, etc). La segunda tiene que ver más con el framework Expo, que permite manejar el ciclo completo de desarrollo del APK. Sus ventajas son el hot reloading, que permite actualizar en caliente modificaciones que se vayan haciendo, sin necesidad de recompilar, y además la capacidad de compilar en la nube con EAS (expo application services), lo que acelera el prototipado rápido conveniente para el plazo de entrega del MVP (producto mínimo viable).

API: [Nestjs](#).

NestJS es el núcleo del backend. Se eligió por su arquitectura modular y opinada, basada en los patrones de inyección de dependencias, decoradores y separación por módulos, controladores y servicios, lo cual impone estructura de forma natural y facilita la colaboración en equipo. Adicionalmente, es conveniente su integración nativa con WebSockets mediante adaptadores para Socket.io y su compatibilidad directa con TypeScript.

Websockets: [Socket.io](#).

Socket.io se integra directamente como adaptador de gateway en NestJS y añade reconexión automática, gestión de salas (rooms) para los chats grupales y compatibilidad con entornos donde WebSockets puros pueden fallar.

Bases de Datos: [PostgreSQL](#), [Meilisearch](#).

PostgreSQL es la base de datos principal. Los datos de la plataforma (usuarios, publicaciones, comentarios, likes, favoritos, relaciones de seguimiento, reviews, verificaciones de staff) son inherentemente relacionales y se favorecen de un esquema relacional. Meilisearch se incorpora como motor de búsqueda dedicado. La búsqueda por palabras clave en descripciones, etiquetas, hashtags y perfiles de juegos es un requisito explícito que PostgreSQL podría cubrir con búsqueda de texto completo, pero a medida que crece el volumen de datos conviene tener una BD dedicada a esto. Meilisearch resuelve esto con indexación optimizada con latencias de respuesta de pocos milisegundos.

Cache: [Valkey](#).

Valkey (fork de Redis) opera como capa de memoria estructurada con diversas responsabilidades: almacena caché de lecturas frecuentes, centraliza las sesiones de usuario, permitiendo validar

autenticación en WebSockets y REST sin consultar PostgreSQL en cada request. Actúa como capa de sincronización entre datos, con BullMQ, maneja la cola de trabajos asíncronos (notificaciones, emails, entre otros).

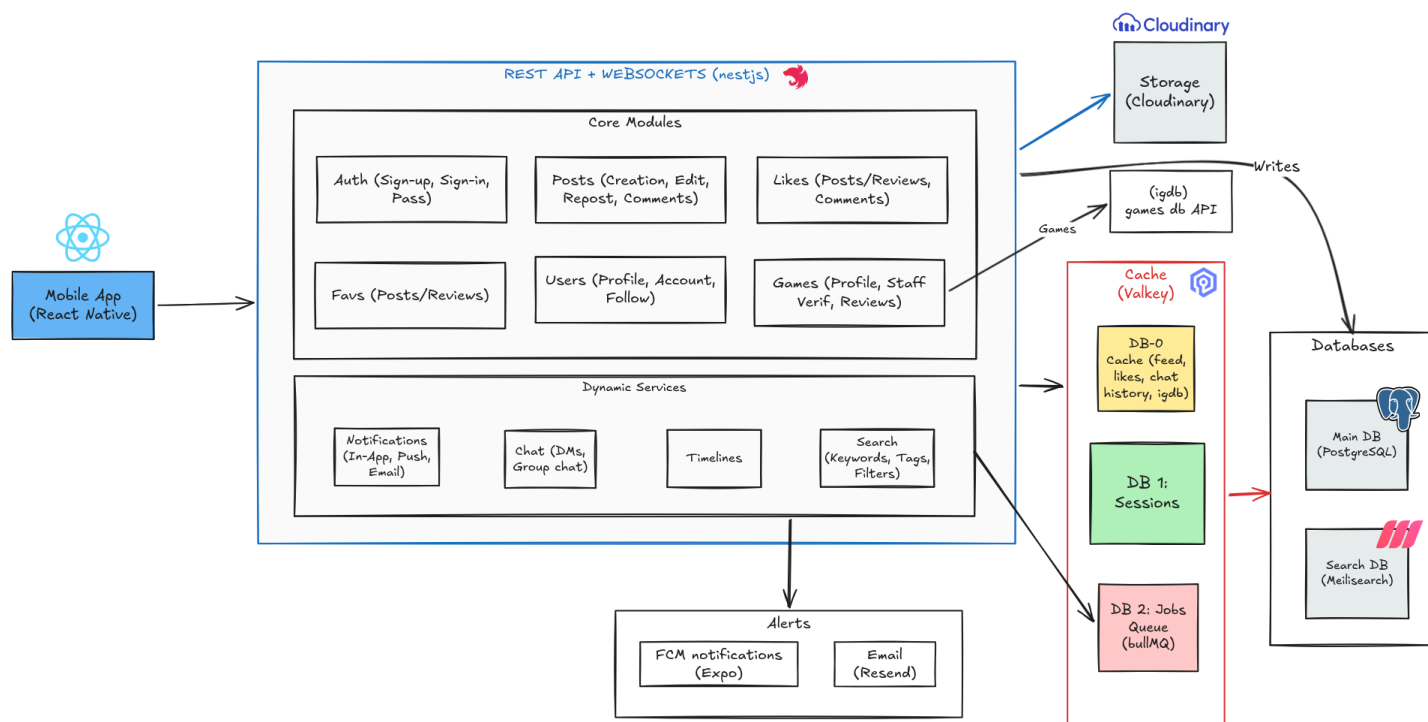
Almacenamiento de multimedia: [Cloudinary](#).

Todo el contenido multimedia generado por usuarios (fotos de publicaciones, avatares) se externaliza a Cloudinary. Esto libera al servidor de gestionar almacenamiento de archivos, transformaciones de imagen y CDN.

Otros servicios: [IGDB](#), [Expo FCM notifications](#), [Resend](#).

IGDB provee la base de datos de videojuegos como fuente externa para los perfiles de juego. Expo FCM Notifications gestiona el envío de push notifications al cliente móvil, integrado con Firebase Cloud Messaging sin necesidad de infraestructura adicional. Resend maneja el envío de emails transaccionales (recuperación de contraseña, verificaciones).

Arquitectura de la aplicación.



[Diagrama Arquitectura \(excalidraw\)](#)

Detalles

Valkey:

- DB-0 almacena caché de lecturas frecuentes: feed, conteo de likes, mensajes e historial de chats.
- DB-1 centraliza las sesiones de usuario, permitiendo validar autenticación en WebSockets y REST sin consultar PostgreSQL en cada request.
- DB-2 sostiene BullMQ, la cola de trabajos asíncronos transversal al backend: notificaciones, emails, indexación en Meilisearch, limpieza de sesiones expiradas y expiración de la ventana de edición de posts.

Autenticación y sesiones: El registro e inicio de sesión generan una sesión (JWT) que se guarda en Valkey DB-1. Toda validación de identidad posterior, consulta Valkey sin tocar PostgreSQL, con el objetivo de mantener la velocidad de respuesta dado que PostgreSQL estará haciendo muchas otras operaciones a la vez. La recuperación de contraseña se agrega como job en la cola de BullMQ hacia Resend.

Publicación de contenido y multimedia: Al crear un post, primero se completa el upload a Cloudinary; el cliente sube las imágenes directamente sin pasar por el servidor. Una vez completado el upload, el cliente envía el payload final (URLs, descripción, etiquetas, hashtags) que el servidor persiste en PostgreSQL. Al momento de creación de posts se encola en BullMQ un job para indexar el post en Meilisearch.

Generación de feed: Al publicar o repostear, el servidor ejecuta fan-out on write: distribuye el post al feed pre-computado de cada seguidor en Valkey DB 0. Un repost se persiste en PostgreSQL con referencia al post original y al usuario que reposteo, y entra al fan-out como cualquier post nuevo; el cliente es quien diferencia la renderización.

Búsqueda: Las búsquedas por palabras clave, etiquetas y hashtags van directamente a Meilisearch, que mantiene un índice de posts, usuarios, comentarios y perfiles de juegos con tolerancia a errores tipográficos y soporte de filtros compuestos. La sincronización del índice se maneja como job en

BullMQ disparado en cada creación, edición o eliminación de contenido indexable, con Valkey DB 2 como mecanismo de coordinación para actualizaciones incrementales. IGDB provee los datos base de los perfiles de juego,.

Chat en tiempo real: La negociación WebSocket se autentica contra Valkey DB 1. Los mensajes se distribuyen en tiempo real a través de rooms de Socket.io y se escriben en un buffer en Valkey por conversación. Un job periódico en BullMQ escribe lo que tiene el buffer a PostgreSQL en lotes. Al abrir una conversación el historial reciente se sirve desde Valkey; el scroll hacia mensajes anteriores no cacheados página directamente sobre PostgreSQL.

Notificaciones: Eventos como likes, comentarios, reposts y nuevos seguidores generan jobs en BullMQ sin bloquear el flujo principal que los originó. Los workers consumen la cola y despachan por canales según el tipo (in-app, FCM).

Prototipo visual

El prototipo de la aplicación se desarrolló en Figma, una herramienta práctica y fácil de usar, ideal para diseñar aplicaciones completas.

Enlace al Prototipo

<https://www.figma.com/design/JYdiHOGm0ENknYF7M3ASWV/Videojuego?node-id=0-1&t=5V9CaE04Rs7Wrvot-1>